

AI ASSISTED CODING

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

NAME : G.Vijaya Simha Reddy

ROLLNO:2503A52671

BATCH:23

Task 1: Password Strength Validator

Requirements: Minimum 8 characters, uppercase, lowercase, digit, special character, and no spaces.

```
def is_strong_password(password):  
    if len(password) < 8:  
        return False  
    if " " in password:  
        return False  
  
    has_upper = False  
    has_lower = False  
    has_digit = False  
    has_special = False  
  
    special = "!@#$%^&*()-_+=[]{}|\\:;\"<>,.?/"
```



```
    for ch in password:  
        if ch.isupper():  
            has_upper = True  
        elif ch.islower():  
            has_lower = True
```

```
elif ch.isdigit():  
    has_digit = True  
elif ch in special:  
    has_special = True
```

```
return has_upper and has_lower and has_digit and has_special
```

Assert Test Cases

```
assert is_strong_password("Abcd@123") == True  
assert is_strong_password("abcd123") == False  
assert is_strong_password("ABcd@1234") == True  
assert is_strong_password("Abc 123@") == False
```

```
print("Task 1 Passed")
```

Output:

Task 1 Passed

Task 2: Number Classification using Loops

Requirements: Classify Positive, Negative, Zero, and handle invalid inputs like strings and None

```
def classify_number(n):  
    if n is None or isinstance(n, str):  
        return "Invalid Input"  
  
    result = ""  
  
    for _ in range(1):    # loop used as required
```

```
if n > 0:
    result = "Positive"
elif n < 0:
    result = "Negative"
else:
    result = "Zero"

return result
```

Assert Test Cases

```
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(-1) == "Negative"
assert classify_number(1) == "Positive"
assert classify_number("abc") == "Invalid Input"
assert classify_number(None) == "Invalid Input"

print("Task 2 Passed")
```

Ouput:

Task 2 Passed

Task 3: Anagram Checker

Requirements: Ignore case, spaces, punctuation, and handle empty strings.

```
def clean(text):
    result = ""
```

```
for ch in text.lower():
    if ch.isalnum():
        result += ch
return result
```

```
def is_anagram(str1, str2):
    return sorted(clean(str1)) == sorted(clean(str2))
```

Assert Test Cases

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("", "") == True
assert is_anagram("Madam", "Damam") == True
```

```
print("Task 3 Passed")
```

Output:

Task 3 Passed

Task 4: Inventory Class

Requirements: Create methods `add_item()`, `remove_item()`, and `get_stock()`.

```
class Inventory:
    def __init__(self):
        self.stock = {}
```

```
def add_item(self, name, quantity):
```

```
    if name in self.stock:
```

```
        self.stock[name] += quantity
```

```
    else:
```

```
        self.stock[name] = quantity
```

```
def remove_item(self, name, quantity):
```

```
    if name in self.stock:
```

```
        self.stock[name] = max(0, self.stock[name] - quantity)
```

```
def get_stock(self, name):
```

```
    return self.stock.get(name, 0)
```

```
# Assert Test Cases
```

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

```
inv.remove_item("Book", 5)
```

```
assert inv.get_stock("Book") == 0
```

```
print("Task 4 Passed")
```

Output:

Task 4 Passed

Task 5: Date Validation & Formatting

Requirements: Accept MM/DD/YYYY, reject invalid dates, and convert to YYYY-MM-DD.

```
from datetime import datetime
```

```
def validate_and_format_date(date_str):  
    try:  
        date = datetime.strptime(date_str, "%m/%d/%Y")  
        return date.strftime("%Y-%m-%d")  
    except:  
        return "Invalid Date"
```

Assert Test Cases

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"  
assert validate_and_format_date("02/30/2023") == "Invalid Date"  
assert validate_and_format_date("01/01/2024") == "2024-01-01"  
assert validate_and_format_date("13/01/2024") == "Invalid Date"
```

```
print("Task 5 Passed")
```

Output:

Task 5 Passed